Projet Python : classification de dessins

Classification supervisée et optimisation

ENSTA – OM2-GM01

Elise Grosjean

Démarrage : Git

Avant de commencer : créer le dépôt Git

Objectif

Un dépôt Git partagé par binôme, avec des commits réguliers et un historique lisible.

Option conseillée : dépôt déjà créé

- Créer un dépôt vide sur GitHub/GitLab.
- Cocher éventuellement `Add README`.
- Copier l'URL du dépôt.

```
git clone URL_DU_DEPOT
cd projet-python
```

Si le dossier local existe déjà

```
mkdir projet-python
cd projet-python
git init
git branch -M main
git remote add origin URL_DU_DEPOT
git pull origin main
```

Placer ensuite le notebook du projet dans le dossier, puis effectuer le premier commit.

Git pendant le projet

Cycle de travail à chaque séance

```
git pull
git status
git add .
git commit -m "Message court et informatif"
git push
```

Fichiers à ne pas versionner : `.gitignore`

```
quickdraw_cache/
.ipynb_checkpoints/
__pycache__/
*.pyc
.DS_Store
```

- Le notebook, le `README.md` et les fichiers `.py`/`.tex` doivent être versionnés.
- Éviter le commit unique à la fin : l'historique fait partie du rendu.

Plan du projet

1 Démarrage : Git

2 Introduction

3 Générer et afficher les données

4 construire le classifieur

5 Optimiser la log-loss

6 Évaluer et comprendre le modèle

7 Défi : dessinez vous-même

8 Bonus et bilan

Introduction

Problème étudié

Construire un classifieur binaire probabiliste

capable de distinguer deux catégories de dessins à main levée.

Trois couples au choix

main / pied **pingouin / perroquet** **oignon / ananas**

- Première catégorie du couple : $y = 0$.
- Seconde catégorie du couple : $y = 1$.
- Question préambule : classer intuitivement les trois couples du plus facile au plus difficile.

Compétences travaillées

Python

- `NumPy` : tableaux, vectorisation, produit matriciel.
- `Matplotlib` : figures, courbes, images.
- `Qt` : traitement d'images.
- `PIL` : conversion de dessins en images.
- `scikit-learn` : séparation train/test et comparaison finale.
- Fonctions, listes, dictionnaires, boucles, conditions.

Mathématiques numériques

- Classification supervisée binaire.
- Log-loss et régularisation.
- Gradient et différences finies.
- Descente de gradient à pas fixe.
- Recherche linéaire d'Armijo.
- Extensions : Newton, SGD, réseau de neurones.

De Quick, Draw ! à un problème mathématique

Chaque dessin est converti en image 28×28, puis aplati en vecteur :

$$\text{dessin} \longrightarrow X_{\text{img}} \in [0, 255]^{28\times 28} \longrightarrow x \in [0, 1]^{784}.$$

Classification supervisée

On dispose d'observations $(x_i, y_i)_{1\leq i\leq n}$, avec

$$x_i \in \mathbb{R}^{784}, \qquad y_i \in \{0, 1\}.$$

Objectif : prédire l'étiquette d'un dessin non vu pendant l'apprentissage.

- Le code de téléchargement est fourni.
- Le coeur du projet est l'apprentissage de la régression logistique par optimisation.

Générer et afficher les données

Choix du couple et chargement

Dans le notebook

$$\texttt{PAIR} \in \{\texttt{main-pied}, \texttt{pingouin-perroquet}, \texttt{oignon-ananas}\}.$$

[2]:
```
## 1/ Choisissez votre problème ici

PAIR = "..."  # "main-pied", "pingouin-perroquet", "oignon-ananas"
N_PER_CLASS = ...

PAIRS = ...

cat0, cat1, nom0, nom1 = PAIRS[PAIR]
print(f"Problème choisi : {nom0} (Étiquette 0) / {nom1} (Étiquette 1)")
```

```
Problème choisi : main (Étiquette 0) / pied (Étiquette 1)
```

Étape 1 : Convertir un dessin vectoriel QuickDraw en bitmap centré size × size

```
# e.g. strokes = [
#    [10, 15, 21, 27],   # coordonnes x
#    [30, 25, 22, 20]    # coordonnes y
#    ] pour reprsenter les traits P1=[10,30] -> P2=[15,25] -> P3
 =[21,22] -> P4=[27,20]

# on les convertit en tableau de pixels de taille size*size avec les
  entiers 0 ou 255: noir = 0 /blanc = 255
# avec size = taille finale de l'image
# et margin = espace autour du dessin lors du recentrage
```

```
      img = Image.new("L", (size, size), 0) # cration image PIL : "L"
  = image en niveaux de gris et 0 = image noire
        ax.imshow(Ximg[idx], cmap="gray", vmin=0, vmax=255)
o Ximg est un bitmap
```

```python
def image_converter(strokes, size=28, margin=2, radius=1.0):
    """
    Convertit un dessin vectoriel QuickDraw en bitmap centré ``size x size``.

    Un dessin QuickDraw est une liste de traits. Chaque trait contient
    une liste de coordonnées x et une liste de coordonnées y.

    Exemple
    -------
    stroke = [
        [10, 15, 21, 27],   # coordonnées x
        [30, 25, 22, 20]    # coordonnées y
    ]

    Cela représente la ligne brisée
    P1=(10,30) -> P2=(15,25) -> P3=(21,22) -> P4=(27,20).

    Retour
    ------
    numpy.ndarray de forme (size, size), avec :
        0   = fond noir,
        255 = trait blanc.
    """

    # ------------------------------------------------------------
    # 1. Rassembler tous les points du dessin
    # ------------------------------------------------------------
    pts = []
    for stroke in strokes:
        xs, ys = stroke[0], stroke[1]
        pts.extend(zip(xs, ys))

    if not pts:
        return np.zeros((size, size), dtype=np.uint8)

    xs = np.array([p[0] for p in pts], dtype=float)
```

```
# ------------------------------------------------
# On réutilisera cette fonction ensuite dans prepare
# pour traiter un dessin réalisé à la souris ou cha
scale, offx, offy = compute_centering_parameters(
    width, height, size=size, margin=margin
)

bitmap = np.zeros((size, size), dtype=float)
# La transformation appliquée à chaque point est :
#
# x_final = offx + (x - xmin) * scale
# y_final = offy + (y - ymin) * scale
#
# (x - xmin, y - ymin) ramène d'abord la boîte englo
# scale change sa taille ;
# offx et offy la placent au centre de l'image.

# ------------------------------------------------
# 4. Dessiner les traits dans l'image finale
# ------------------------------------------------
for stroke in strokes:
    sx, sy = stroke[0], stroke[1]

    points = [
        (
```

```
83  def paint_segment(bitmap, p0, p1, radius):
84      """
85      Relie les points p0 et p1 par un trait continu.
86
87      p0 = (x0, y0)
88      p1 = (x1, y1)
89      """
90
91      x0, y0 = p0
92      x1, y1 = p1
93
94      # TODO
95
96      # 1. Calculer la distance entre p0 et p1.
97      #
98      distance = ...
99
100     # 2. Choisir la distance maximale entre deux centres
101     #    de disques successifs.
102     step = ...
103
104     # 3. Déterminer le nombre de points intermédiaires en utilisant ceil.
105     #
106     n = max(2, int(...))
107
108     # 4. Construire les coordonnées interpolées avec np.linspace.
109     #
110     xs = ...
```

```python
## Pour tester vos fonctions
def test_paint_disk():

    bitmap = np.zeros((28, 28), dtype=float)

    paint_disk(
        bitmap,
        x=14,
        y=14,
        radius=2
    )

    plt.figure(figsize=(4, 4))
    plt.imshow(
        bitmap,
        cmap="gray",
        vmin=0,
        vmax=1
    )

    plt.title("Test de paint_disk")
    plt.axis("off")
    plt.show()


def test_paint_segment():

    bitmap = np.zeros((28, 28), dtype=float)

    paint_segment(
        bitmap,
        p0=(1, 20),
        p1=(7, 15),
        radius=2
    )
```

Redimensionner et recentrer un dessin

On veut transformer le dessin original en une image 28×28.

Variables géométriques

- `size` : taille de l'image finale. Ici, `size = 28`.
- `margin` : marge que l'on garde autour du dessin. Ici, `margin = 2`.
- `usable` : longueur disponible entre les deux marges.
- `width`, `height` : largeur et hauteur du dessin original.

Les centres des pixels sont indexés de 0 à 27, donc la longueur totale en coordonnées est 27 et non 28 .

Choix du facteur d'échelle.
La plus grande dimension doit occuper toute la longueur disponible :

$$\max(\texttt{width}, \texttt{height}) \times \texttt{scale} = \texttt{usable}$$

donc

$$\texttt{scale} = \frac{\texttt{usable}}{\max(\texttt{width}, \texttt{height})}.$$

$$\texttt{scaled_width} = \texttt{width} \times \texttt{scale}$$

Idem pour `scaled_height`

On veut transformer le dessin original en une image 28×28.

```
# ----- En termes de PIXELS -----
#
#  0  1 |  2  3  4  5 | 6 ... 21 | 22 23 24 25 | 26 27
# ------|-------------|----------|-------------|------
# marge |    libre    |  dessin  |    libre    | marge
# 2 pix |    4 pix    | 16 pix   |    4 pix    | 2 pix


# ----- En termes de LONGUEURS entre centres de pixels -----
#
# 0            2         6                  21        25          27
# |------------|---------|------------------|---------|-----------|
#     marge       libre       dessin           libre      marge
#       2           4            15              4          2
```

Ainsi, avec une marge `margin = 2` de chaque côté,

$$\texttt{usable} = \texttt{size} - 1 - 2\,\texttt{margin} = 28 - 1 - 4 = 23.$$

À retenir

28 pixels $\Longrightarrow$ 27 intervalles entre leurs centres

Dans cet exemple :

$$\texttt{scaled_width} = 15, \qquad \texttt{free_space} = 23 - 15 = 8.$$

Pour centrer, on met la moitié de l'espace libre à gauche et l'autre moitié à droite :

$$\texttt{offx} = \texttt{margin} + \frac{\texttt{usable} - \texttt{scaled_width}}{2}.$$

Même principe verticalement pour `offy` (mais ici `usable = scaled_height`).

...

```
offx + (x - xmin) * scale
```

Fonction à compléter

Exercice : compléter les lignes marquées TODO.

```
def compute_centering_parameters(width, height,
                                 size=28, margin=2):
    """
    Calcule les parametres de redimensionnement et de recentrage.

    Retour :
        scale : facteur d'echelle
        offx  : decalage horizontal
        offy  : decalage vertical
    """

    # Eviter les divisions par zero

    # Longueur disponible entre les deux marges.
    # Attention : les pixels sont indexes de 0 a size-1.
    usable = ...  # TODO

    ...

    return scale, offx, offy
```

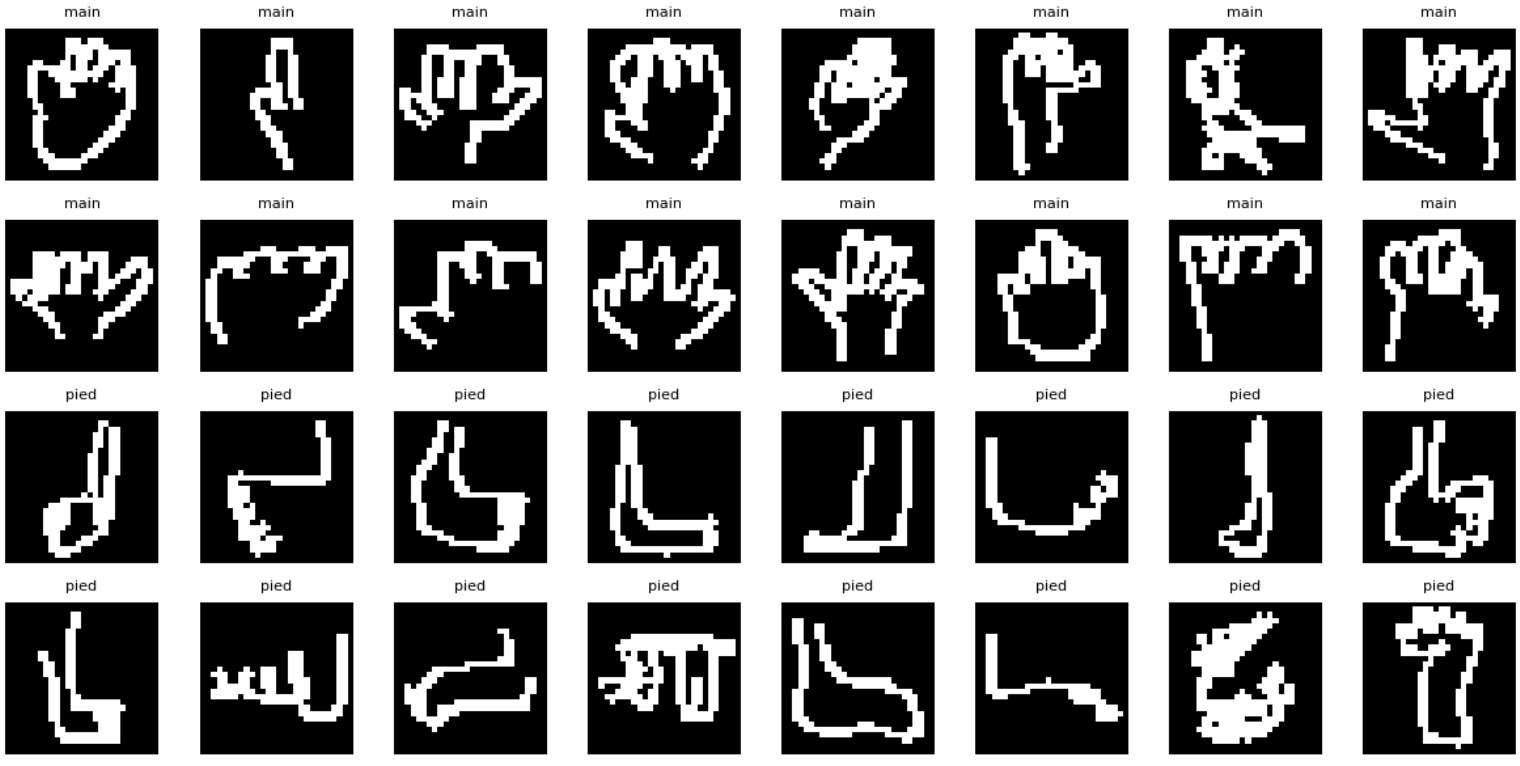
Choix du couple et chargement

- Charger le même nombre de dessins par classe.
- Vérifier les effectifs avec `np.bincount(y)`.
- Afficher plusieurs exemples de chaque catégorie.

Questions

- Combien d'observations dans chaque classe ?
- Quelle est la dimension avant et après aplatissement ?
- Quelles parties des dessins semblent discriminantes ?

Premières visualisations



- Deux dessins d'une même classe peuvent être éloignés dans l'espace des pixels.
- Variabilité : orientation, taille, position, style, niveau de détail.

construire le classifieur

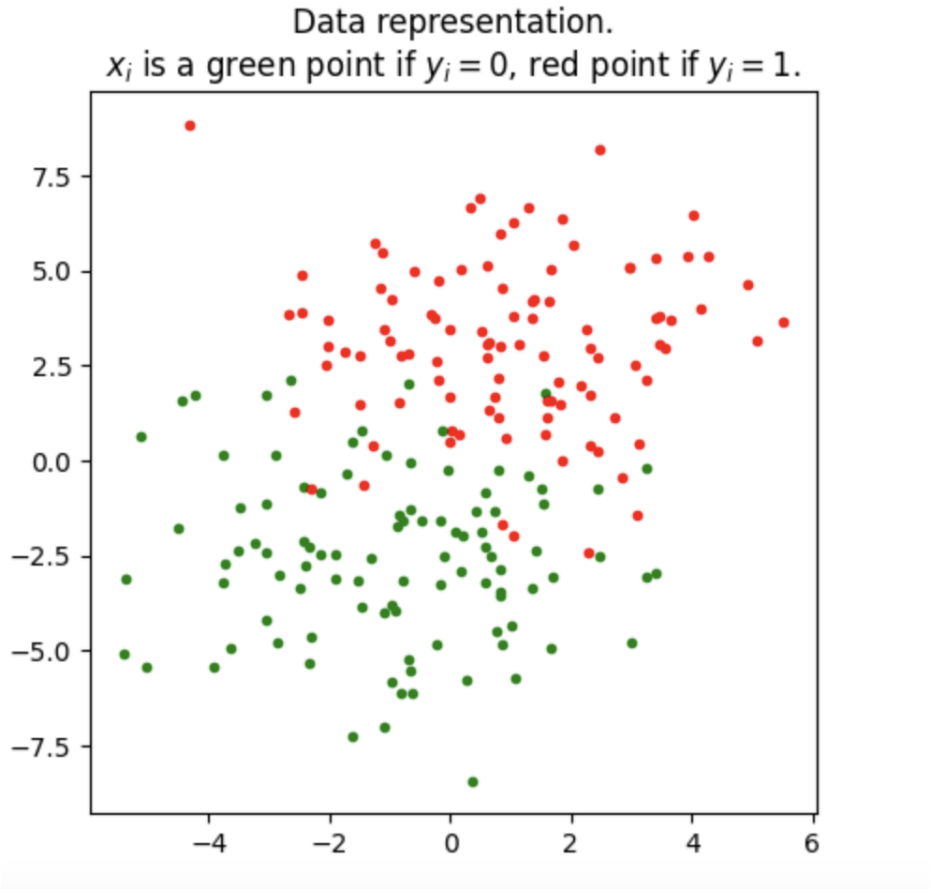
Un exemple de classification en dimension 2

Idée simplifiée

Supposons que chaque observation soit décrite par deux caractéristiques :

$$x_i = (x_{i,1}, x_{i,2}) \in \mathbb{R}^2.$$

Les points verts ont l'étiquette $y_i = 0$; les points rouges ont l'étiquette $y_i = 1$.



- But : apprendre une séparation entre les deux classes.
- Dans le projet, la dimension est 784, pas 2.

Frontière de décision

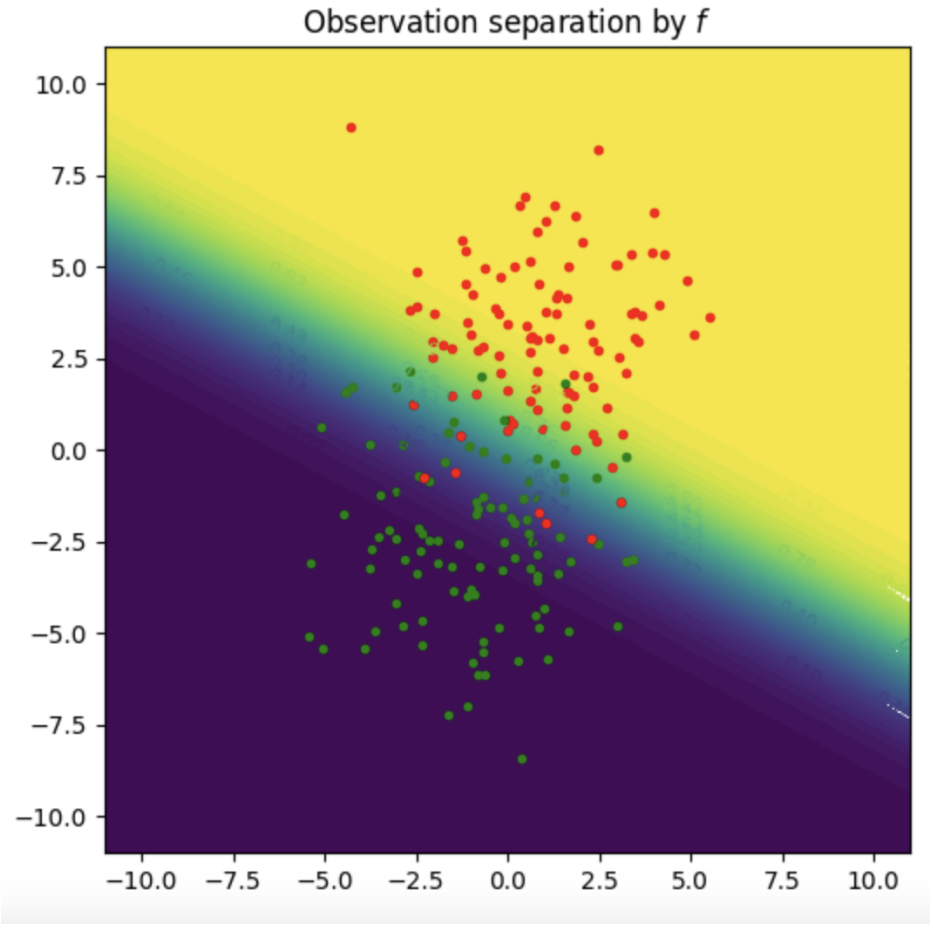
Classifieur linéaire

On cherche $w \in \mathbb{R}^d$ et $b \in \mathbb{R}$ tels que

$$w^T x + b = 0.$$

En dimension 2, c'est une droite. En dimension 784, c'est un hyperplan.

- D'un côté de l'hyperplan : prédiction $y = 0$.
- De l'autre côté : prédiction $y = 1$.

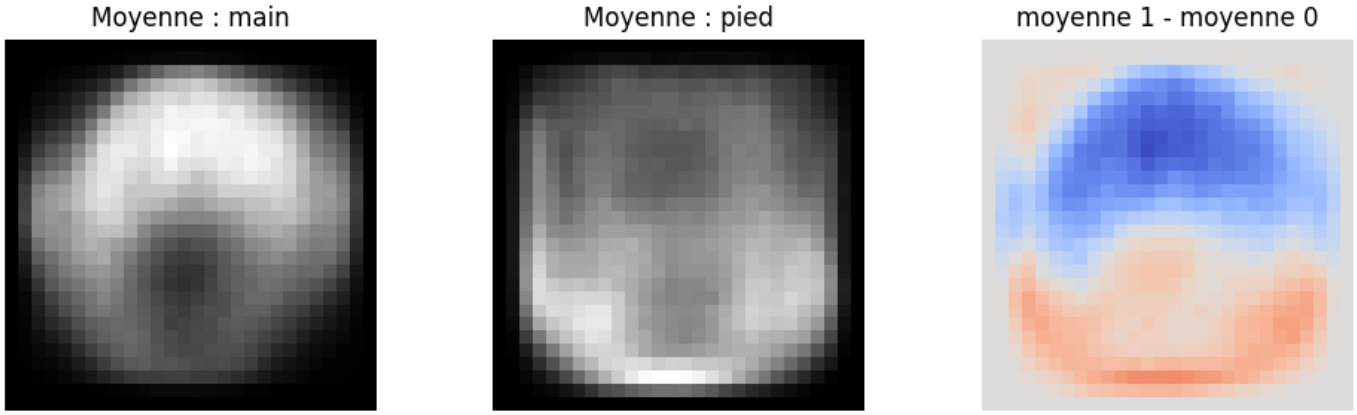


Images moyennes

Pour chaque classe, on calcule une image moyenne :

$$\bar{x}_0 = \frac{1}{n_0} \sum_{i:y_i=0} x_i, \qquad \bar{x}_1 = \frac{1}{n_1} \sum_{i:y_i=1} x_i.$$

On affiche aussi la différence $\bar{x}_1 - \bar{x}_0$.



- Les moyennes sont floues car les dessins ne sont pas parfaitement alignés.

Prétraitement

Normalisation

Les pixels sont ramenés de $[0, 255]$ vers $[0, 1]$:

$$X = \frac{X_{\text{img}}}{255}.$$

Amplitudes plus modérées, stabilise les méthodes de gradient.

Aplatissement et séparation

$28 \times 28 \longrightarrow 784$, 70% apprentissage, 30% test.

- Le jeu d'apprentissage sert à déterminer les paramètres.
- Le jeu de test sert uniquement à évaluer la généralisation.

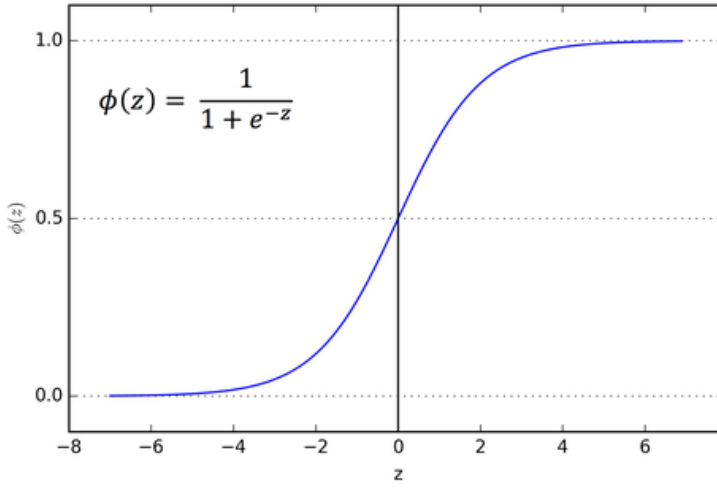
Score linéaire et sigmoïde

Pour un dessin $x \in \mathbb{R}^{784}$, on définit

$$z = w^T x + b, \qquad w \in \mathbb{R}^{784}, \quad b \in \mathbb{R}.$$

Le score est transformé en probabilité par

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad P(y=1 \mid x) = \sigma(w^T x + b).$$



Règle de décision

$$\sigma(w^T x + b) \geq \frac{1}{2} \iff w^T x + b \geq 0 : y = 1$$

$$\sigma(w^T x + b) < \frac{1}{2} \iff w^T x + b < 0 : y = 0$$

Biais inclus dans les variables

Pour simplifier le code, on augmente la dimension :

$$\widetilde{x} = \begin{pmatrix} x \\ 1 \end{pmatrix} \in \mathbb{R}^{785}, \qquad \theta = \begin{pmatrix} w \\ b \end{pmatrix} \in \mathbb{R}^{785}.$$

Alors

$$w^T x + b = \theta^T \widetilde{x}.$$

Avantage pratique

Une seule variable `theta` contient tous les paramètres, y compris le biais.

Attention

On régularise w, mais pas le biais b : le dernier coefficient de régularisation est donc nul.

Log-loss régularisée

Pour

$$p_i = \sigma(\theta^T \widetilde{x}_i),$$

on minimise

$$f(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\Big(y_i \log p_i + (1-y_i)\log(1-p_i)\Big) + \frac{\lambda}{2}\|w\|^2.$$

Règle de décision

$$\sigma(w^T x + b) \geq \frac{1}{2} \iff w^T x + b \geq 0 : y = 1$$

$$\sigma(w^T x + b) < \frac{1}{2} \iff w^T x + b < 0 : y = 0$$

Interprétation

- Une bonne prédiction confiante donne une faible contribution.
- Une mauvaise prédiction confiante est fortement pénalisée.
- λ contrôle la taille des poids.

Pourquoi le gradient ?

On veut minimiser la log-loss :

$$\min_{\theta \in \mathbb{R}^{785}} f(\theta).$$

Pour une fonction différentiable, un minimum local intérieur θ^* vérifie, sous des hypothèses usuelles,

$$\nabla f(\theta^*) = 0.$$

Idée numérique

On ne résout pas directement cette équation : on construit une suite (θ_k) qui fait décroître f.

$$\theta_{k+1} = \theta_k + \alpha_k d_k.$$

La direction d_k et le pas α_k déterminent la qualité de l'algorithme.

Directions de descente

Soit $f : \mathbb{R}^m \to \mathbb{R}$ différentiable. Pour α petit,

$$f(x + \alpha d) = f(x) + \alpha \nabla f(x)^T d + o(\alpha).$$

Définition

Un vecteur d est une direction de descente en x si

$$\nabla f(x)^T d < 0.$$

Alors, pour $\alpha > 0$ assez petit,

$$f(x + \alpha d) < f(x).$$

Choix naturel

Si $\nabla f(x) \neq 0$, alors

$$d = -\nabla f(x), \qquad \nabla f(x)^T d = -\|\nabla f(x)\|^2 < 0.$$

Gradient de la log-loss

On calcule donc les gradients :
On veut obtenir, sans régularisation,

$$\nabla f(\theta) = \frac{1}{n}\widetilde{X}^T(p - y),$$

et avec régularisation

$$\nabla f(\theta) = \frac{1}{n}\widetilde{X}^T(p - y) + \lambda \begin{pmatrix} w \\ 0 \end{pmatrix}.$$

À faire dans le notebook

- Retrouver ces formules avec $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- Vérifier les dimensions : $\widetilde{X} \in \mathbb{R}^{n \times 785}$ et $p - y \in \mathbb{R}^n$.
- Programmer `objective` et `grad_objective`.

Idée des différences finies

On approxime une dérivée en observant la variation de la fonction autour d'un point.

$$f'(\theta) = \lim_{h \to 0} \frac{f(\theta + h) - f(\theta)}{h}.$$

ou

$$f'(\theta) \simeq \frac{f(\theta + h) - f(\theta - h)}{2h}.$$

Pour une fonction de plusieurs variables, on ne modifie qu'une coordonnée :

$$\frac{\partial f}{\partial \theta_j}(\theta) \simeq \frac{f(\theta + he_j) - f(\theta - he_j)}{2h}.$$

$$\nabla f_{\text{analytique}}(\theta) \approx \nabla f_{\text{numérique}}(\theta).$$

Vérification par différences finies

Avant d'optimiser, on vérifie le gradient analytique :

$$\frac{\partial f}{\partial \theta_j}(\theta) \simeq \frac{f(\theta + he_j) - f(\theta - he_j)}{2h}.$$

Questions

- Quel comportement attendre lorsque h diminue ?
- Pourquoi un h extrêmement petit n'est-il pas toujours meilleur ?
- Pourquoi cette étape est-elle utile avant la descente de gradient ?

Un gradient faux peut donner une descente de gradient totalement trompeuse.

Optimiser la log-loss

Descente de gradient à pas fixe

On utilise l'itération

$$\theta_{k+1} = \theta_k - \alpha \nabla f(\theta_k).$$

À programmer

Une fonction renvoyant :

- la solution finale θ ;
- l'historique des valeurs $f(\theta_k)$;
- l'historique des normes $\|\nabla f(\theta_k)\|$;
- le nombre d'itérations, le nombre d'évaluations de f, le temps CPU.

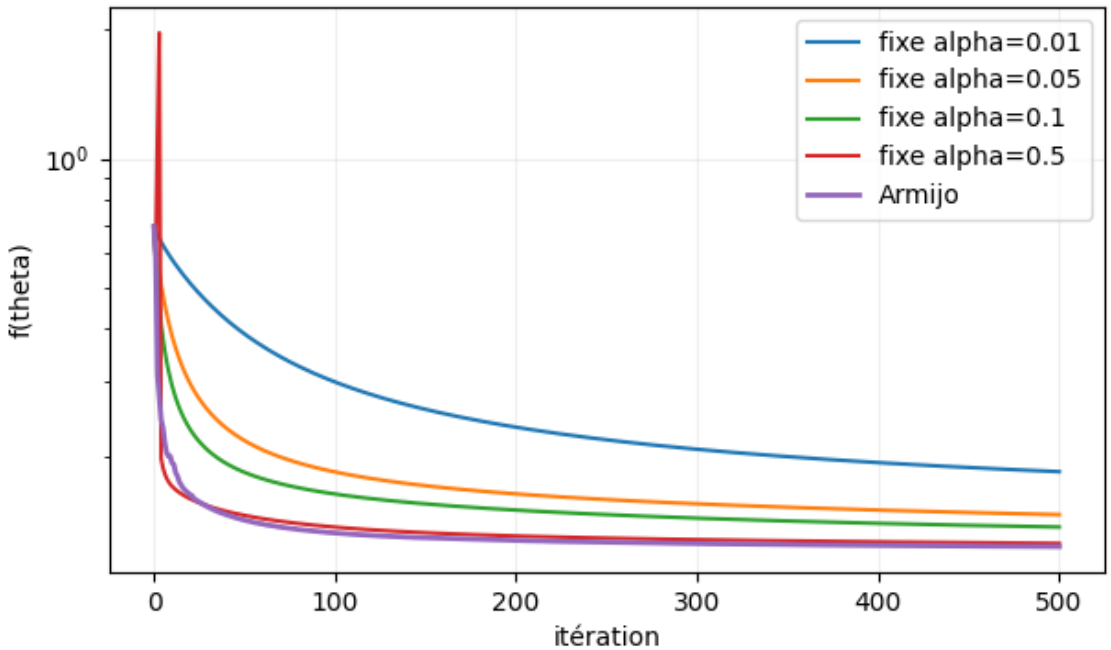
Question clé

Comment choisir α ? Un pas trop petit ralentit ; un pas trop grand peut faire diverger.

Influence du pas

Tester plusieurs valeurs de α :

$$10^{-3}, \quad 10^{-2}, \quad 10^{-1}, \quad 0.5, \quad 1, \quad 2.$$



- Pas trop petit : convergence lente.
- Pas trop grand : oscillations ou divergence possibles.
- Le meilleur pas peut dépendre du couple choisi.

Évaluer et comprendre le modèle

Prédictions sur le jeu de test

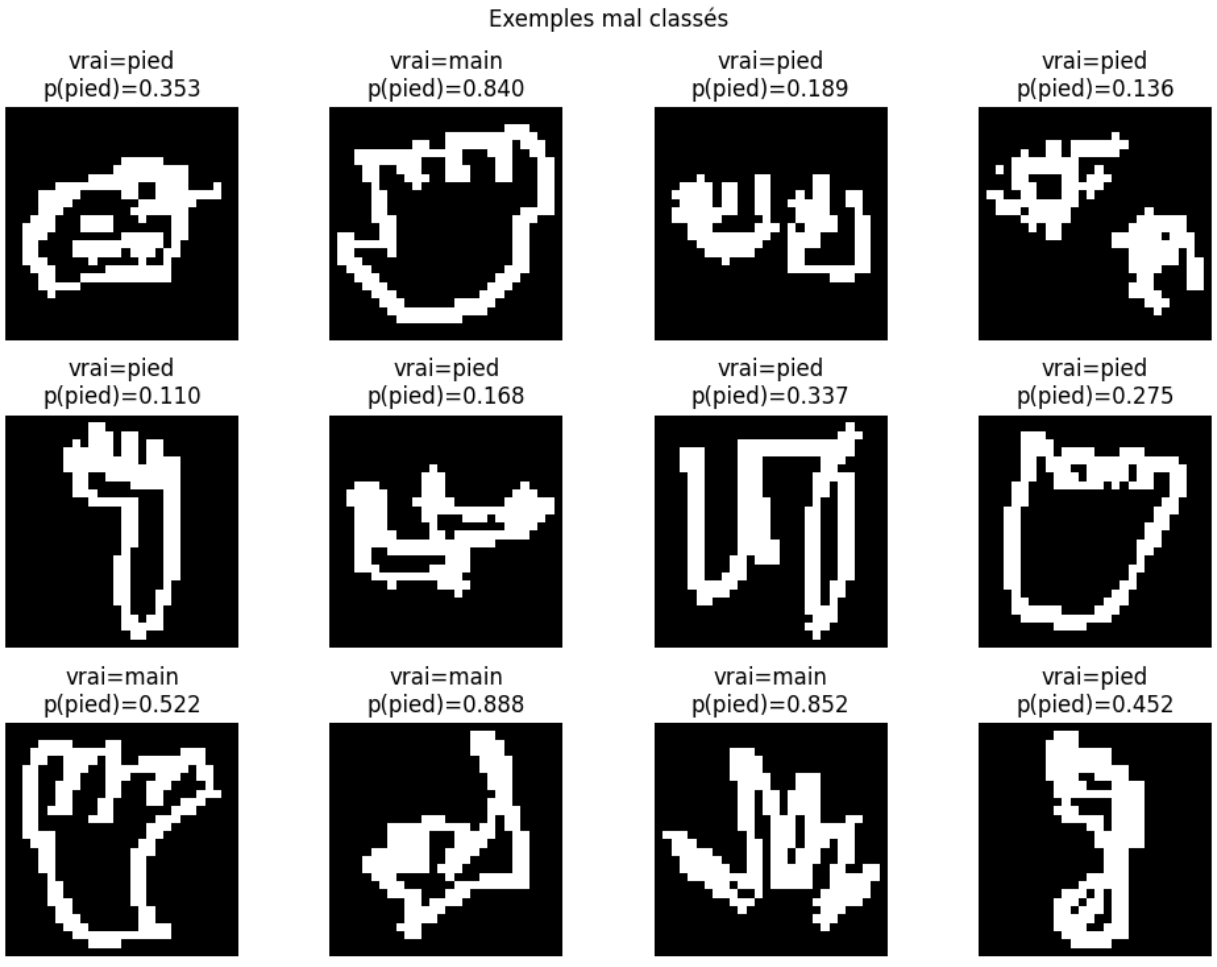
À partir de $\theta^\star$, pour chaque observation de test :

$$p_i = \sigma\big((\theta^\star)^T \widetilde{x}_i\big), \qquad \widehat{y}_i = \mathbf{1}_{p_i \geq 0.5}.$$

Mesures demandées

- Accuracy : proportion de prédictions correctes.
- Precision classe 1 : parmi les prédictions 1, proportion de vrais 1.
- Recall classe 1 : parmi les vrais 1, proportion retrouvée.
- Log-loss : qualité des probabilités prédites.
- Matrice de confusion : vrais/faux positifs et négatifs.

Dessins mal classés

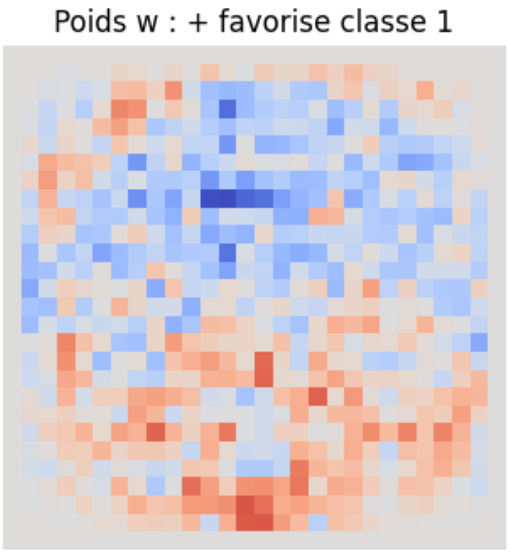


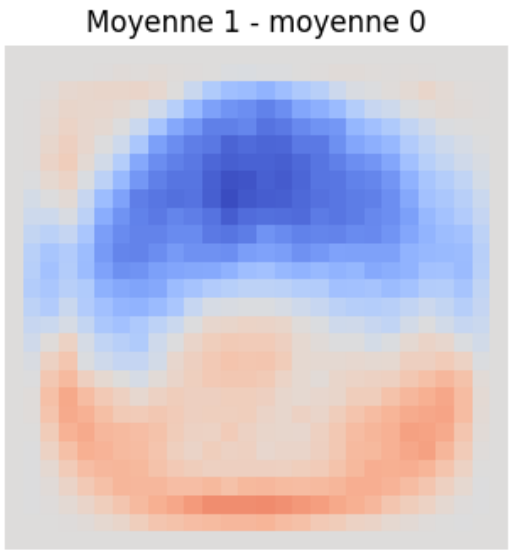
Questions

- Peut-on expliquer visuellement certaines erreurs ?

Visualiser le vecteur w

Le vecteur appris $w \in \mathbb{R}^{784}$ peut être remis sous forme d'image 28×28.





- Coefficients positifs : pixels favorisant la classe 1.
- Coefficients négatifs : pixels favorisant la classe 0.
- À comparer avec $\bar{x}_1 - \bar{x}_0$.

Comparaison avec `scikit-learn`

À la fin seulement, utiliser `sklearn.linear_model.LogisticRegression` comme référence.

À comparer

- accuracy et log-loss sur le test ;
- matrice de confusion ;
- coefficients appris ;
- temps de calcul.

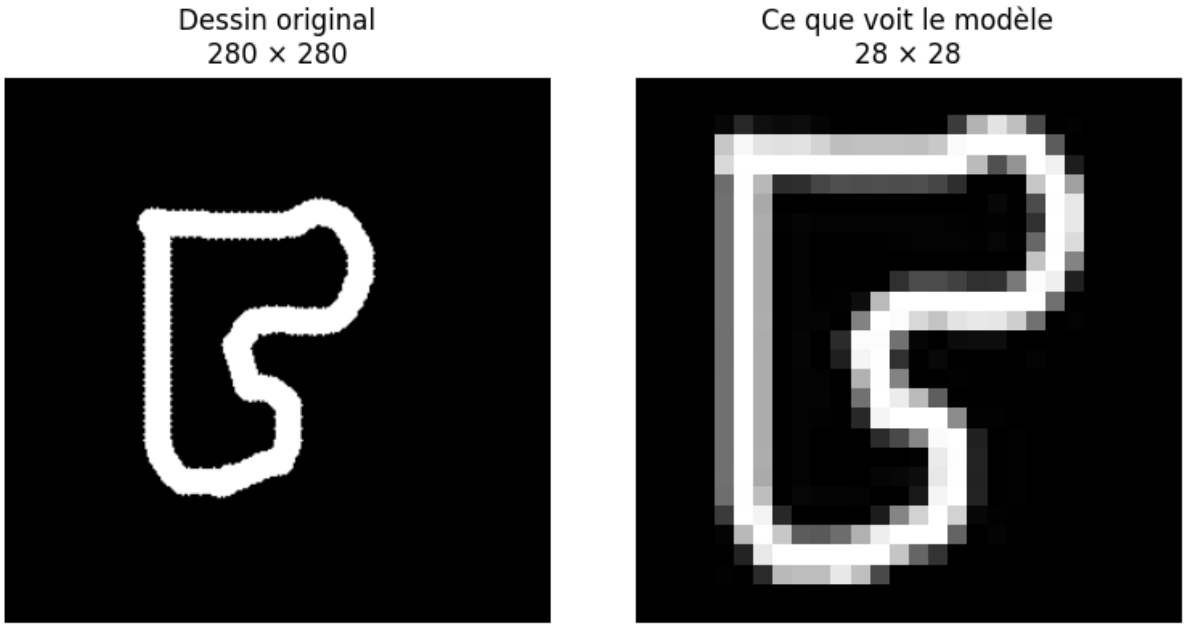
Attention

Une différence de performance ne signifie pas automatiquement qu'un algorithme d'optimisation est meilleur : les conventions de régularisation et les objectifs peuvent différer.

Défi : dessinez vous-même

Test sur vos propres dessins

Le notebook contient un canevas interactif. Si le backend interactif n'est pas disponible, vous pouvez charger un fichier PNG.



Pipeline appliqué à votre dessin

dessin personnel $280 \times 280 \longrightarrow$ recentrage $\longrightarrow 28 \times 28 \longrightarrow P(y = 1 \mid x)$.

Expériences demandées

1. Dessiner un objet de chacune des deux classes, sans chercher à aider le modèle.
2. Dessiner avec le moins de traits possible, tout en restant reconnaissable pour un humain.
3. Produire volontairement un dessin possédant des caractéristiques des deux classes.
4. Tenter d'obtenir une prédiction incorrecte avec forte confiance, sans modifier le code.

Questions finales

- Quelles parties semblent indispensables pour votre modèle ?
- Votre dessin ambigu l'est-il aussi pour un humain ?
- Le modèle met-il en évidence un problème de données, de modèle, ou les deux ?

Bonus et bilan

Extensions possibles

Règle de Wolfe

Implémenter une recherche linéaire satisfaisant les conditions de Wolfe et la comparer à Armijo.

Méthode de Newton

Pour la régression logistique :

$$\nabla^2 f(\theta) = \frac{1}{n}\widetilde{X}^T D \widetilde{X} + \lambda R, \qquad D = \mathrm{diag}(p_i(1-p_i)).$$

Résoudre un système linéaire plutôt que calculer explicitement l'inverse.

Autres extensions

- Comparer les trois couples : main/pied, pingouin/perroquet, oignon/ananas.
- Tester un modèle non linéaire, par exemple un petit réseau de neurones.
- Ajouter une version mini-batch ou gradient stochastique.

Principe

Toute amélioration doit être mesurée sur le jeu de test, jamais uniquement sur les données d'apprentissage.

Bilan

Image $\rightarrow$ pixels $\rightarrow$ modèle probabiliste

$\rightarrow$ optimisation $\rightarrow$ prédiction $\rightarrow$ interprétation.

À la fin du projet : revenir au classement intuitif des couples et le comparer aux performances mesurées.

Règle d'Armijo

Pour $d_k = -\nabla f(\theta_k)$, on cherche $\alpha > 0$ tel que

$$f(\theta_k + \alpha d_k) \leq f(\theta_k) + c_1 \alpha \nabla f(\theta_k)^T d_k.$$

Backtracking

On part de α_0, puis tant que la condition échoue :

$$\alpha \leftarrow \rho\alpha, \qquad 0 < \rho < 1.$$

- c_1 fixe l'exigence de décroissance.
- ρ fixe la vitesse de réduction du pas.
- Armijo évite de choisir un pas fixe a priori.

Algorithme de descente de gradient

Structure générale

1. Choisir θ_0, une tolérance ε et un nombre maximal d'itérations.
2. Tant que $\|\nabla f(\theta_k)\| \geq \varepsilon$:
   1. calculer $g_k = \nabla f(\theta_k)$;
   2. choisir $d_k = -g_k$;
   3. choisir α_k : pas fixe ou Armijo ;
   4. poser $\theta_{k+1} = \theta_k + \alpha_k d_k$.
3. Retourner θ_k et les historiques numériques.

Le pas est crucial : il contrôle à la fois stabilité et vitesse.

Comparaison numérique

Comparer pas fixe et Armijo selon :

- nombre d'itérations ;
- nombre d'évaluations de f ;
- temps CPU ;
- valeur finale de f ;
- norme finale du gradient.

Question importante

Une méthode utilisant moins d'itérations est-elle nécessairement plus rapide ?

Une itération peut être plus coûteuse si elle contient plusieurs évaluations de f.